



Hyperparameter Exploration in CNNs

Name	ID
Raghad Mohmed	20220704
Youssef Nabil	20221205
Youssef Mohamed	20220405
Nour-aldeen Alaa	20220360
Norhan Yasser	20220370

Introduction

In this report, we explore different configurations and models on the MNIST dataset.

We begin with a simple SVM model & ANN, then we try a CNN. At each step we change some hyperparameters (layers, dropouts, learning rates, etc) and try to see the impact & differences (regarding accuracy, time taken for training, computational resources & memory efficiency) at each step.

At the end, we reach the best configurations of a CNN model for the MNIST dataset, after studying and knowing the impact of each hyperparameter.

What is the MNIST Dataset?

The MNIST dataset (Modified National Institute of Standards and Technology) consists of grayscale images of handwritten digits ranging from 0 to 9.

- Each image is 28×28 pixels, resulting in 784 features per image when flattened into a vector.
- The dataset includes 60,000 training samples and 10,000 test samples.
Its simplicity and well-labeled structure make it ideal for experimenting with classification algorithms.

SVM: Model (1)

What is a Support Vector Machine (SVM)?

Support Vector Machine is a supervised learning algorithm commonly used for classification tasks. The key idea behind SVM is to find the optimal boundary (or hyperplane) that separates data points belonging to different classes with the maximum margin.

SVM can also handle non-linearly separable data using kernel functions, which transform the input space to make it easier to draw separating boundaries. A commonly used kernel is the Radial Basis Function (RBF), which allows SVM to classify more complex patterns.

Using SVM for Image Classification

Although SVM wasn't originally designed for image data, it can still perform well when images are converted into feature vectors. In the case of MNIST:

- Each image is reshaped into a 784-dimensional vector.
- The SVM model then learns to recognize digit patterns based on these pixel values.
- Preprocessing, such as feature scaling, significantly improves the model's performance and speed.

Observations

- While SVM performs well on MNIST, it may take longer to train compared to other models (like neural networks), especially on large datasets.
- Kernel used: RBF (Radial Basis Function)
- C value: 5 (controls trade-off between smooth decision boundary and classifying training points correctly)
- Execution time: 5 minutes and 49 seconds
- Test Accuracy: 98%
- Dataset used: Full MNIST dataset (60,000 training / 10,000 test images)

ANN: Model(2)

```
#Sequential -> left to right

model_2 = Sequential(name="model_2")

model_2.add(Flatten(input_shape=(28, 28))) #Input

model_2.add(Dense(128, activation='relu')) #Hidden layer 1

model_2.add(Dense(64, activation='relu')) #Hidden layer 2

model_2.add(Dense(10, activation='softmax')) #Output

model_2.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

model_2.fit(X_train, y_train, epochs=10, batch_size=64 , validation_split=0.1 , shuffle=True)
```

Accuracy after 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	81.87%	95.72%

2	95.68%	96.85%
3	97.02%	97.02%
4	97.83%	96.97%
5	98.29%	97.62%

Final accuracy	96.85%
Number of parameters	Between input & first hidden layer = $(784 \times 128) + 128 = 100480$ Between the first & second hidden layer = $(128 \times 64) + 64 = 8256$ Between the last hidden layer and the output layer = $(10 \times 64) + 10 = 650$ Total = $100480 + 8256 + 650 = 109386$
Average train time	3.93 Sec
Total test time	2.24 Sec
Layers	1. Flatten 784 2. Dense #1 128 -> RELU 3. Dense #2 64 -> RELU 4. Output 10-> Softmax
Learning rate	By default, with Adam optimizer, $lr = 0.001$
Optimizer used	Adam

Observations

- I trained the model for 10 epochs. After about 5 epochs, the accuracy reached over **97%**, and improvements became smaller, showing the model was converging.
- The training accuracy reached **99.5%**, while the test accuracy was around **96.8%**, this shows a slight overfitting.

Conclusion

10 epochs were sufficient for convergence, more than 10 would overfit the model without giving better results. Also, Adam optimizer with $lr = 0.001$ is known to perform well on MNIST.

Initial CNN: Model(3)

```
# 1. Create the model

model_3 = Sequential(name='model_3')

# 2.Convolutional layer

model_3.add(Conv2D(32, (3, 3), input_shape=(28, 28, 1)))

# 3. Activation function

model_3.add(Activation('relu'))

# 4. Pooling

model_3.add(MaxPooling2D(pool_size=(2, 2)))

# 5. Flatten the image to 1D prepare it for the dense (Fully Connected) layer

model_3.add(Flatten())

model_3.add(Dense(128))

model_3.add(Activation('relu'))

# 6. Output Layer

model_3.add(Dense(10))

model_3.add(Activation('softmax'))

# Optimizer

sgd = SGD(momentum=0.9, learning_rate=0.01)

model_3.compile(optimizer=sgd, loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

model_3.fit(X_train_conv, y_train, epochs=1, shuffle=True, batch_size=32,
validation_split=0.1)
```

Epoch number	Training accuracy	Validation accuracy
1	83.04%	96.17%

Accuracy	96.71%
Final accuracy	96.71% (same since it is only one epoch)
Number of parameters	Input layer: 28x28x1 # params = 0 Conv2D: 32(3x3)x1 # params = 32(3x3)+32= 320 □ 26x26x32 Activation: 26x26x32 # params = 0 Pooling : 2x2 □ 13x13x32 # params = 0 Flatten: 5408 # params = 0 □ Dense(128): 128 # params = (5408x128)+128= 692,352 Activation: 128 # params = 0 □ Dense(10): 10 # params = (128x10)+10 = 1,290 ∴ Total = 320 + 692,352 + 1,290 = 693,962
Average train time	38sec
Average test time	4.11sec
Layers	1. Conv2D + ReLU 2. Pooling layer 3. Flatten layer 4. Dense layer (128) + ReLU 5. Dense layer (10) + softmax
Learning rate	0.01
Optimizer used	SGD (Stochastic Gradient Descent)

Observations

- The model achieved a test accuracy of 95.60% after just one epoch of training.
- This relatively high performance can be attributed to the simplicity of the MNIST dataset, which consists of clean, well-structured images of handwritten digits.

- Even with minimal training, the model was able to learn key features necessary for accurate classification.

Conclusion

To evaluate the model's baseline performance before any tuning, we limited training to one epoch. Despite this. However, by applying hyperparameter tuning (e.g., increasing epochs, adjusting learning rates, using regularization), we expect to achieve even higher accuracy and generalization.

Epoch Tuning

Model(4) : number of epochs = 5

```
model_4.fit(X_train_conv, y_train, validation_split=0.1, epochs= 5, batch_size= 32, shuffle=True)
```

Accuracy in first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	82.44%	95.55%
2	95.57%	96.68%
3	96.82%	97.33%
4	97.41%	97.73%
5	98.06%	97.90%

Final accuracy	97.90%
Number of parameters	693,962 (no change)
Average train time	7.78 sec
Total test time	1.44 sec
Layers	5 layers (no change)

Learning rate	0.01 (no change)
Optimizer used	SGD

Observations

- Increasing the number of training epochs resulted in a noticeable improvement in both validation accuracy during the final epoch and test accuracy, which reached 97.51%.
- As expected, this improvement came with a trade-off: longer training time due to the additional epochs.
- The model continued to learn effectively beyond the first epoch, indicating that it had not yet overfit and could still generalize well to unseen data.

Conclusion

Increasing the number of epochs led to a clear improvement in model accuracy. This suggests that the model benefits from additional training cycles on the MNIST dataset.

Given this positive trend, we plan to continue experimenting with more epochs to further explore the model's potential performance — while also monitoring for signs of overfitting or diminishing returns.

Model(5) : number of epochs = 10

```
model_5.fit(X_train_conv, y_train, validation_split=0.1, epochs= 10, batch_size= 32, shuffle=
True)
```

Accuracy after 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	82.37%	95.43%
2	95.44%	96.93%
3	96.81%	95.82%
4	97.39%	97.78%
5	97.99%	97.68%

Final accuracy	98.18%
Number of parameters	693,962 (no change)
Average train time	5.33 sec
Total test time	2.4 sec
Layers	5 layers (no change)
Learning rate	0.01 (no change)
Optimizer used	SGD

Observations

- After adjusting the training parameters, the model's performance continued to improve.
- Validation accuracy reached 98% in the final epoch, with test accuracy also achieving 98%, indicating strong generalization.

Conclusion

Increasing the number of epochs significantly enhanced the model's performance. However, to balance training time and accuracy, we chose to stick with 10 epochs. This setting provides a high-performing yet efficient model, avoiding the risks of overfitting and longer training times associated with excessive epochs.

learning rates Tuning

Model (6): learning rate = 0.1

```
sgd = SGD(momentum=0.9, learning_rate=0.1)

model_6.compile(optimizer=sgd, loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

Accuracy after 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	86%	96%
2	95.81%	95.9%
3	96.68%	96.88%
4	96.6%	95.9%
5	97.64%	96.2%

Final accuracy	95.85%
Number of parameters	693,962
Average train time	6.34 sec
Total test time	0.76 sec
Layers	5 layers (no change)
Learning rate	0.1
Optimizer used	SGD

Observations

- There's some fluctuations on the accuracy, it keeps on changing from 96% to 97%, the maximum it reached was in epoch 7: 97.23%, which is still lower than the original 0.01 learning rate which achieved 98.32% accuracy.
- It seems the learning rate of 0.1 is a big number for SGD optimizer, which makes it miss the optimal minimum. That's why the accuracy keeps on changing between two boundaries and not as high as the original learning rate we used.

Conclusion

- Learning rate of 0.1 isn't a good match with SGD, so we'll try a smaller learning rate, we already tried 0.01 so let's check 0.0001.

Model (7): learning rate = 0.0001

```
sgd = SGD(momentum=0.9, learning_rate=0.0001)

model_7.compile(optimizer=sgd, loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

Accuracy after 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	28.67%	72.05%
2	71.77%	83.13%
3	80.33%	86.95%
4	84.54%	88.85%
5	86.81%	89.75%

Final accuracy	90.77%
Number of parameters	693,962
Average train time	5.27 sec
Total test time	0.73 sec
Layers	5 layers (no change)
Learning rate	0.0001
Optimizer used	SGD

Observations

- The model started learning more slowly
- Accuracy in the first 5 epochs was lower than before

- The training was more stable, but way too slower.
- The final accuracy lower
- I think this happened because the model was making smaller updates, so it needed more epochs to learn on it to reach high accuracy, but doing that will result in more time.

Conclusion

- When decreasing the learning rate, the model learns steadily but takes more time and needs more epochs, so when trying to increase the learning rate a bit (0.1) the model kept missing the optimal minimum that's why the training wasn't steady, it was making fluctuations and keep on changing (high to low or low to high) at each step.
- Given what was said above, the best learning rate is 0.01, since it balances both the time taken and the stability of the training and also it gave the highest accuracy so far. (98.32%)

Model Structure Tuning

Conv2D layers tuning

Model(8): 2Conv 2D

```
model_8 = models.Sequential()

# First Conv Layer + MaxPooling

model_8.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model_8.add(layers.MaxPooling2D((2, 2)))

# Second Conv Layer

model_8.add(layers.Conv2D(64, (3, 3), activation='relu'))

model_8.add(layers.MaxPooling2D((2, 2)))

# Flatten

model_8.add(layers.Flatten())

model_8.add(layers.Dense(128, activation='relu'))
```

```

model_8.add(layers.Dense(10, activation='softmax')) # Output layer

# Compile the model

model_8.compile(

    optimizer=SGD(momentum=0.9, learning_rate=0.01),

    loss='sparse_categorical_crossentropy',

    metrics=['accuracy']

)

model_8.fit(X_train_conv, y_train, epochs=10, shuffle=True, batch_size=32,
validation_split=0.1)

```

Accuracy after 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	80.16%	97.83%
2	97.35%	98.32%
3	98.26%	98.37%
4	98.77%	98.53%
5	98.83%	98.82%

Final accuracy	98.84%
----------------	--------

Number of parameters	- Between input & first Conv2D layer = $(3 \times 3 \times 1 + 1) \times 32 = 320$ - Between MaxPooling & second Conv2D layer = $(3 \times 3 \times 32 + 1) \times 64 = 18,496$ - Between Flatten & first Dense layer = $(5 \times 5 \times 64 + 1) \times 128 = 204,928$ - Between first Dense & output Dense layer = $(128 + 1) \times 10 = 1,290$ Total = $320 + 18,496 + 204,928 + 1,290 = 225,034$
Average train time	6.41 Sec
Total test time	0.75 Sec
Layers	1. Conv2D layer #1 with 32 filters, kernel = $3 \times 3 \rightarrow \text{ReLU}$ 2. MaxPooling2D, pool size = 2×2 3. Conv2D layer #2 with 64 filters, kernel = $3 \times 3 \rightarrow \text{ReLU}$ 4. MaxPooling2D, pool size = 2×2 5. Flatten layer 6. Fully Connected (FC) layer #1 with 128 neurons $\rightarrow \text{ReLU}$ 7. Output layer with 10 neurons $\rightarrow \text{Softmax}$
Learning rate	0.01
Optimizer used	SGD

Observations

- The model started slower with an accuracy of 80% and quickly improved, reaching a high test accuracy of **99%** after 10 epochs, which shows it learned from the data well.
- Validation accuracy remained consistently high (98.6% to 98.9%), showing the model generalizes well without overfitting.
- The average training time is 6.41 seconds per epoch.

Conclusion

- The 2Conv2D layers effectively extract spatial features from the images. MaxPooling reduces spatial size and computation, helping generalization.
- The dense layers learn complex combinations of these features for classification.

Model (9):3 Conv2D

```
model_9 = models.Sequential()

# First Conv Layer + MaxPooling

model_9.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model_9.add(layers.MaxPooling2D((2, 2)))

#second conv layer

model_9.add(layers.Conv2D(64, (3, 3), activation='relu'))

model_9.add(layers.MaxPooling2D((2, 2)))

#third conv layer

model_9.add(layers.Conv2D(64, (3, 3), activation='relu'))

model_9.add(layers.MaxPooling2D((2, 2)))

# Flatten and Fully Connected Layers

model_9.add(layers.Flatten())

model_9.add(layers.Dense(128, activation='relu'))    # First FC layer

model_9.add(layers.Dense(10, activation='softmax'))  #(output)

model_9.summary()

# 4. Compile model
```

```

model_9.compile(optimizer='sgd',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

# Time for training

start_train_time = time.time()

model_9.fit(X_train_conv, y_train,epochs=10 ,shuffle=True, batch_size=32,
validation_split=0.1)

```

Accuracy after 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	31.49%	84.12%
2	87.56%	91.78%
3	92.94%	94.92%
4	94.51%	95.65%
5	95.38%	95.88%

Final accuracy	97.63%
Number of parameters	Between input & first conv layer: $(3 \times 3 \times 1 + 1) \times 32 = (9 + 1) \times 32 = 320$ Between first maxPooling & second conv layer: $(3 \times 3 \times 32 + 1) \times 64 = (288 + 1) \times 64 = 18,496$ Between second maxPooling & third conv layer: $(3 \times 3 \times 64 + 1) \times 64 = (576 + 1) \times 64 = 36,928$ Between flatten and first dense layer: $(1 \times 1 \times 64 + 1) \times 128 = (64 + 1) \times 128 = 8,320$ Final dense layer: $(128 + 1) \times 10 = 1,290$ Total = 320 + 18,496 + 36,928 + 8,320 + 1,290 = 65,354
Average train time	8.87 Sec

Total test time	0.76 Sec
Layers	1. Conv2D layer #1 -> ReLU 2. MaxPooling2D 3. Conv2D layer #2 -> ReLU 4. MaxPooling2D 5. Conv2D layer #3 -> ReLU 6. MaxPooling2D 7. Flatten layer 8. Fully connected layer #1 with 128 neurons -> ReLU 9. Output -> Softmax
Learning rate	0.01
Optimizer used	SGD

Observations

- The training started with low accuracy 31% at the first epoch, indicating the model initially struggled to learn but improved quickly.
- Accuracy reaches about 97.63%, which is good but slightly lower than the 2 Conv2D model with 1 Dense layer
- Training time is longer, around 89 seconds compared the the simpler 2 Conv2D
- Adding a third Conv2D layer seems to add complexity but does not improve the accuracy compared to 2 model 8.

Conclusion

The best value here is using 2 Conv2D layers (Model 8) instead of 3 because It achieved a higher accuracy of 99%. Training time is less because adding more layers increased training time, but did not improve accuracy.

FC Layers tuning

Model (10): 2 Conv2D + 2 FC layers

```
model_10 = models.Sequential()

# First Conv Layer + MaxPooling

model_10.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model_10.add(layers.MaxPooling2D((2, 2)))

# second conv layer + MaxPooling

model_10.add(layers.Conv2D(64, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model_10.add(layers.MaxPooling2D((2, 2)))

# Flatten and Fully Connected Layers

model_10.add(layers.Flatten())

model_10.add(layers.Dense(128, activation='relu'))    # First FC layer

model_10.add(layers.Dense(64, activation='relu'))    # Second FC layer

model_10.add(layers.Dense(10, activation='softmax')) # (output)

# 4. Compile model

model_10.compile(optimizer='sgd', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model_10.fit(X_train_conv, y_train, epochs=10, shuffle=True, batch_size=32,
validation_split=0.1)
```

Accuracy after 5 epochs

Epoch number	Training accuracy	Validation accuracy
--------------	-------------------	---------------------

1	52.74%	93.40%
2	92.38%	94.95%
3	95%	94.68%
4	96.20%	97.22%
5	96.62%	97.52%

Final accuracy	98.26%
Number of parameters	Between input & first conv layer: $(3 \times 3 \times 1 + 1) \times 32 = (9 + 1) \times 32 = 320$ Between first maxPooling & second conv layer: $(3 \times 3 \times 32 + 1) \times 64 = (288 + 1) \times 64 = 18,496$ Between Flatten and first Dense layer: $(1600 + 1) \times 128 = 204,928$ Between first Dense and second Dense layer: $(128 + 1) \times 64 = 8,256$ Final dense layer: $(64 + 1) \times 10 = 650$ Total = 320 + 18,496 + 204,928 + 650 = 232,650
Average train time	6.91 Sec
Total test time	0.76 Sec
Layers	1. Conv2D layer #1 -> ReLU 2. MaxPooling2D 3. Conv2D layer #2 -> ReLU 4. MaxPooling2D 5. Flatten layer 6. Fully connected layer #1 with 128 neurons -> ReLU 7. Fully connected layer #2 with 64 neurons -> ReLU 8. Output -> Softmax
Learning rate	0.01
Optimizer used	SGD

Conclusion

- Training time is 69 seconds total, faster than the previous
- Test accuracy = 98.26, slightly less than model 8 but better than model 9.

Model (11): 2 Conv2D + No dense layer

```
model_11 = Sequential()

model_11.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model_11.add(MaxPooling2D((2, 2)))

model_11.add(Conv2D(64, (3, 3), activation='relu'))

model_11.add(MaxPooling2D((2, 2)))

model_11.add(Flatten())

model_11.add(Dense(10, activation='softmax'))

model_11.compile(optimizer=sgd, loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

model_11.fit( X_train_conv, y_train, epochs=10, batch_size=32, validation_split=0.1,
shuffle= True)
```

Accuracy after 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	78.73	97.13%
2	97%	97.93%
3	97.76%	98.22%

4	98.16%	98.42%
5	98.55%	98.43%

Final accuracy	98.57%
Number of parameters	Between input & first conv layer: $(3 \times 3 \times 1 + 1) \times 32 = (9 + 1) \times 32 = 320$ Between first maxPooling & second conv layer: $(3 \times 3 \times 32 + 1) \times 64 = (288 + 1) \times 64 = 18,496$ Final dense layer: $(1600 + 1) \times 10 = 16,010$ Total = 320 + 18,496 + 16,010 = 34826
Average train time	7.6 Sec
Total test time	0.76 Sec
Layers	1. Conv2D layer #1 -> ReLU 2. MaxPooling2D 3. Conv2D layer #2 -> ReLU 4. MaxPooling2D 5. Flatten layer 6. Output -> Softmax
Learning rate	0.01
Optimizer used	SGD

Observations

- This model is less complex than the previous models; it doesn't use any hidden layers
- Despite being simpler, it achieves high accuracy because our dataset is simple.
- It was the lowest number of parameters among all models tried

Model (12) : Change dense layer size to 64

```
model_12 = Sequential()

model_12.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model_12.add(MaxPooling2D((2, 2)))

model_12.add(Flatten())

model_12.add(Dense(64, activation='relu'))

model_12.add(Dense(10, activation='softmax'))

model_12.compile(optimizer=sgd, loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

model_12.fit(X_train_conv, y_train, epochs=10, batch_size=32, validation_split=0.1,
shuffle= True)
```

Accuracy after 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	80.67%	95.87%
2	95.07%	96.58%
3	96.73%	97.42%
4	97.62%	97.60%
5	98.01%	97.82%

Final accuracy	97.70%
----------------	--------

Number of parameters	Between input & first conv layer: $(3 \times 3 \times 1 + 1) \times 32 = (9 + 1) \times 32 = 320$ Between flatten and first dense : $(5408 + 1) \times 64 = 346,176$ Final dense layer: $(64 + 1) \times 10 = 650$ Total = 320 + 346,176 + 650 = 347,146
Average train time	6.3 Sec
Total test time	0.76 Sec
Layers	1. Conv2D layer #1 -> ReLU 2. MaxPooling2D 3. Flatten layer 4. Dense layer (64) 5. Output -> Softmax
Learning rate	0.01
Optimizer used	SGD

Observation

- Model 8 achieved higher test accuracy (99.02%) than Model 12 (97.70%) with lower test loss (3.72% vs. 7.73%).
- Despite being simpler, **Model 12** has more parameters (347K vs. 225K) due to early flattening.
- Training time per epoch is similar for both.

Conclusion

This study shows that CNN performance depends on finding the right balance between model complexity and efficiency. **Model 8** achieved the highest accuracy of 99% among all 5 models we tested.

Choosing the model with 2 Conv layers gives a better accuracy (than the 1 Conv) and still saves resources (better than using 3 Conv).

Model 11 gave us a high accuracy, proving that the simpler models can still perform very well. Models with extra dense layers did not always improve results, showing that more layers don't always mean better performance. Meanwhile, overly simple models lost accuracy, highlighting the need for enough depth to learn meaningful features.

Batch Size Tuning

Model (13): Batch size = 64

```
model_13.fit(X_train_conv, y_train, epochs=10, shuffle=True, batch_size=64, validation_split=0.1)
```

Accuracy in the first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	37.48%	88.02%
2	87.43%	93.57%
3	92.11%	95.03%
4	94.07%	96.08%
5	95.04%	96.18%

Final accuracy	97.44%
Number of parameters	225,034
Average train time	5.29 sec
Total test time	1.92 sec
Layers	7 layers (no change)
Learning rate	0.01 (no change)
Optimizer used	SGD

Observations

- Rapid improvement in training accuracy shows fast convergence and effective learning with ReLU.
- Increasing batch size to 64 led to faster training time, although test time was slightly longer due to processing larger batches.

Conclusion

Batch size of 64 resulted in faster training time per epoch. And test accuracy decreased (98% -> 97%)

Model (14): Batch size = 128

```
model_14.fit(X_train_conv, y_train, epochs=10, shuffle=True, batch_size=128, validation_split=0.1)
```

Accuracy in first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	22.33%	69.80%
2	72.79%	88.17%
3	86.37%	91.57%
4	89.91%	93.47%
5	91.60%	93.93%

Final accuracy	95.84%
Number of parameters	225,034
Average train time	3.00 sec
Total test time	1.83 sec
Layers	7 layers (no change)
Learning rate	0.01 (no change)
Optimizer used	SGD (Stochastic Gradient Descent)

Observations

- Training speed improved significantly, average training time dropped to 3 sec
- Final accuracy was slightly lower than with smaller batch sizes 32 and 64.

Conclusion

Batch size of 128 offers excellent training speed but it comes with a slight trade-off in final accuracy, which was about 1-2% lower than the models trained with batch size of 32 or 64.

Model (15): Batch size = 250

```
model_13.fit(X_train_conv, y_train, epochs=10, shuffle=True, batch_size=250, validation_split=0.1)
```

Accuracy in first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	24.69%	55.58%
2	58.19%	74.42%
3	76.46%	86.38%
4	84.38%	89.58%
5	87.76%	91.03%

Final accuracy	93.89%
Number of parameters	225,034
Average train time	2.29 sec
Total test time	1.93 sec
Layers	7 layers (no change)

Learning rate	0.01 (no change)
Optimizer used	SGD

Observations

- Increasing batch size to 250 improved training speed but hurt the accuracy.
- Smaller batch sizes gave better generalization on validation and test sets.

Activation Function Tuning

Model (16): Sigmoid

```
model_16 = models.Sequential()

# First Conv Layer + MaxPooling

model_16.add(layers.Conv2D(32, (3, 3), activation='sigmoid', input_shape=(28, 28, 1)))

model_16.add(layers.MaxPooling2D((2, 2)))

# First Conv Layer + MaxPooling

model_16.add(layers.Conv2D(64, (3, 3), activation='sigmoid'))

model_16.add(layers.MaxPooling2D((2, 2)))

# Flatten and Fully Connected Layers

model_16.add(layers.Flatten())

model_16.add(layers.Dense(128, activation='sigmoid'))    # First FC layer

model_16.add(layers.Dense(10, activation='softmax'))    # Second FC layer (output)
```

Accuracy in the first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	10.37%	9.52%
2	11.03%	9.60%
3	10.84%	10.50%
4	10.74%	10.50%
5	10.95%	10.50%

Final accuracy	25.31%
Number of parameters	225,034
Average train time	8.43 sec
Total test time	1.37 sec
Layers	7 layers (no change)
Learning rate	0.01 (no change)
Optimizer used	SGD

Observations

- The model shows very low accuracy on both training and validation sets throughout all epochs.
- The model fails to learn meaningful patterns.

Conclusion

Sigmoid only works with binary classification, so using it with 10 classes will lead to a model that basically randomly guesses the class for the sample. It does normalize each output (making it a value $[0,1]$) but these values won't add up to 1, so the model will be confused which class to choose when there is more than one class with the same probability.

Model (17): Tanh

```
model_17 = models.Sequential()

# First Conv Layer + MaxPooling

model_17.add(layers.Conv2D(32, (3, 3), activation='tanh', input_shape=(28, 28, 1)))

model_17.add(layers.MaxPooling2D((2, 2)))

# Second Conv Layer + MaxPooling

model_17.add(layers.Conv2D(64, (3, 3), activation='tanh'))

model_17.add(layers.MaxPooling2D((2, 2)))

# Flatten and Fully Connected Layers

model_17.add(layers.Flatten())

model_17.add(layers.Dense(128, activation='tanh'))    # First FC layer

model_17.add(layers.Dense(10, activation='softmax'))  # Second FC layer (output)
```

Accuracy in the first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	66.07%	93.03%
2	92.11%	95.42%
3	94.37%	96.20%
4	95.56%	97.08%
5	96.52%	97.38%

Final accuracy	98.16%
Number of parameters	225,034
Average train time	9.02 sec
Total test time	1.13 sec
Layers	7 layers (no change)
Learning rate	0.01 (no change)
Optimizer used	SGD

Observations

- The model trains effectively with Tanh, reaching high accuracy early.
- The test accuracy is excellent, showing the model has learned to classify well.

Conclusion

Tanh performs well in this model, especially in early learning stages, due to centered activation outputs $[-1, 1]$ which can benefit optimization.

Model (18): Swish

```
model_18 = Sequential([

    # First Conv Layer + MaxPooling

    Conv2D(32, (3, 3), activation=swish, input_shape=(28, 28, 1)),

    MaxPooling2D((2, 2)),

    #Second conv layer

    Conv2D(64, (3, 3), activation=swish),

    MaxPooling2D((2, 2)),

    # Flatten and Fully Connected Layers
```

```

Flatten(),

Dense(128, activation=swish),

Dense(10, activation='softmax') # Second FC layer (output)])

```

Accuracy in the first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	16.30%	68.83%
2	78.75%	92.72%
3	92.16%	94.80%
4	94.93%	96.45%
5	95.89%	96.73%

Final accuracy	97.96%
Number of parameters	225,034
Average train time	7.72 sec
Total test time	1.38 sec
Layers	7 layers (no change)
Learning rate	0.01 (no change)
Optimizer used	SGD

Observations

- In the 1st epoch the model started with relatively low training accuracy, which is typical for the Swish due to its smoother non-linearity and delayed convergence.
- Validation accuracy jumped significantly after 1 epoch (from ~68% to ~92%), showing fast generalization.
- By epoch 5 the model performs similarly to the best ReLU/ Tanh versions.
- Slightly faster training than the Tanh model.

Conclusion

Swish activation performs excellently, with very high test accuracy and fast generalization after the first few epochs, it maintains good training efficiency.

Optimizer Tuning

Model (19): Adam optimizer

```
model_19.add(Dense(10,
activation='softmax'))model_19.compile(optimizer=Adam(learning_rate=0.01),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Accuracy in first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	91.99%	97.07%
2	97.27%	98.10%
3	98.04%	97.67%
4	98.18%	98.02%
5	98.20%	98.08%

Final accuracy	98.03%
Number of parameters	225,034
Average train time	7.86 sec

Total test time	1.39 sec
Layers	7 layers (same as model 8)
Learning rate	0.01 (no change)
Optimizer used	Adam

Observation

- The final test accuracy using the Adam optimizer was slightly lower compared to the model using SGD (98.03% vs. 99.02%).
- Additionally, Adam required more total training and testing time.
- While Adam showed faster initial convergence, SGD achieved better overall generalization with lower test loss.

Conclusion

Despite Adam's quick early learning, SGD proved more efficient and accurate in this case, making it the preferred optimizer for this model.

Model (20) : RMSprop optimizer

```
model_20.compile(optimizer=RMSprop(learning_rate=0.01),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Accuracy in first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	88.66%	94.23%
2	97.42%	97.27%
3	97.97%	98.10%
4	97.97%	98.18%
5	98.24%	93.38%

Final accuracy	98.50%
Number of parameters	225,034
Average train time	8.06 sec
Total test time	0.95 sec
Layers	7 layers (same as model 8)
Learning rate	0.01 (no change)
Optimizer used	RSM

Observation

- The model using RMSprop achieved a final test accuracy of 98.50%, which is higher than Adam (98.03%) but still lower than SGD (99.02%).
- The test loss (10.26%) was better than Adam's (11.57%) but notably higher than SGD's (3.72%), indicating less effective generalization than SGD.
- RMSprop also had the longest total training time (80.63s) and a moderate test time (0.95s).

Conclusion

While RMSprop performed better than Adam in both accuracy and loss, it did not surpass SGD in either performance or efficiency. Therefore, SGD remains the most effective optimizer among the three for this CNN model.

Dropout Tuning

Model (21): Dropout=0.15 (after first Conv2D layer)

```
model_21 = models.Sequential()

# First Conv Layer + MaxPooling

model_21.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model_21.add(layers.MaxPooling2D((2,2)))

model_21.add(Dropout(0.15))

# Second Conv Layer + MaxPooling

model_21.add(layers.Conv2D(64, (3, 3), activation='relu'))

model_21.add(layers.MaxPooling2D((2,2)))

# Flatten and Fully Connected Layers

model_21.add(layers.Flatten())

model_21.add(layers.Dense(128, activation='relu'))    # First FC layer

model_21.add(layers.Dense(10, activation='softmax'))  #(output)
```

Accuracy in first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	80.45%	97.52%
2	97.16%	98.07%
3	98.01%	98.38%
4	98.40%	98.47%
5	98.75%	98.58%

Final accuracy	98.82%
Number of parameters	225,034
Average train time	7.77 sec
Total test time	0.78 sec
Layers	1. Conv2D layer #1 with 32 filters, kernel = 3x3 → ReLU 2. MaxPooling2D, pool size = 2x2 3. Dropout layer with rate 0.15 4. Conv2D layer #2 with 64 filters, kernel = 3x3 → ReLU 5. MaxPooling2D, pool size = 2x2 6. Flatten layer 7. Fully Connected (FC) layer #1 with 128 neurons → ReLU 8. Output layer with 10 neurons → Softmax
Learning rate	0.01 (no change)
Optimizer used	SGD

Observation

- We used a dropout with a rate of 0.15 after the first convolutional layer.
- Dropouts are used to prevent overfitting, so we were thinking that it would be more useful if we put the dropout in the early stages of learning, in the first convolutional layer the model starts to learn the patterns and features of the data, so it would be helpful to put the dropout there to prevent any overfitting that might happen from the early stages of learning.
That's what we thought of when we were trying to see where to put the dropout layer.

Conclusion

- No increase in the accuracy, the best accuracy achieved so far was 98.84% with no dropouts, adding a dropout made the accuracy 98.82%, which is no big difference, that might mean that there's no overfitting happens in the data, but we'll check different rates and locations of dropouts to make sure.

Model (22): 2 Dropouts (0.5: after the first Conv layer, 0.8: after dense)

```
model_22 = models.Sequential()

# First Conv Layer + MaxPooling

model_22.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model_22.add(layers.Dropout(0.5))

model_22.add(layers.MaxPooling2D((2, 2)))

#Second Conv layer + MaxPooling

model_22.add(layers.Conv2D(64, (3, 3), activation='relu'))

model_22.add(layers.MaxPooling2D((2, 2)))

# Flatten and Fully Connected Layers

model_22.add(layers.Flatten())

model_22.add(layers.Dense(128, activation='relu'))    # First FC layer

model_22.add(layers.Dropout(0.8))

model_22.add(layers.Dense(10, activation='softmax'))  #(output)
```

Accuracy in first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	39.85%	91.55%
2	79.06%	95.18%

3	84.65%	95.97%
4	87.87%	96.72%
5	89.20%	96.65%

Final accuracy	97.40%
Number of parameters	225,034
Average train time	8.94 sec
Total test time	1.37 sec
Layers	1. Conv2D layer #1 with 32 filters, kernel = 3x3 → ReLU 2. MaxPooling2D, pool size = 2x2 3. Dropout layer with rate 0.5 4. Conv2D layer #2 with 64 filters, kernel = 3x3 → ReLU 5. MaxPooling2D, pool size = 2x2 6. Flatten layer 7. Fully Connected (FC) layer #1 with 128 neurons → ReLU 8. Dropout layer with rate 0.8 9. Output layer with 10 neurons → Softmax
Learning rate	0.01 (no change)
Optimizer used	SGD Optimizer

Dropout Notes

- We used a dropout rate of 0.5 after the first convolutional layer, and another one with rate of 0.8 after the first dense layer.
- The location of the first dropout layer is for the same reason as the previous model, we only increased the rate to see if that would change anything.
- We decided to put another dropout layer after the dense layer since it has many parameters, which means high risk of overfitting, this prevents memorization of the training data.

Observations

- The accuracy decreased (97.40%), probably because of the high rate of dropout after the dense layer, 80% dropout rate means 20% of the units are active at each training step.
- This tells us that increasing the dropout rate isn't necessarily good, and probably a rate of 0.3 ~ 0.5 is good, other than that we'll risk losing too many parameters which will decrease the accuracy.

Conclusion

- We tried to put dropouts to prevent any overfitting, but it seems there's no overfitting happening in the data.
- Adding dropouts didn't affect the accuracy as we expected, it also decreased a little, but no increase.
- Given what was said above, we don't need any dropouts in this data, since no overfitting happens in the regular training with no dropouts.

Best-Performing Configurations

Model(23)

```
model_23= models.Sequential()

# First Conv Layer + MaxPooling

model_23.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model_23.add(layers.MaxPooling2D((2,2)))

# Second Conv Layer + MaxPooling

model_23.add(layers.Conv2D(64, (3, 3), activation='relu'))

model_23.add(layers.MaxPooling2D((2,2)))

# Flatten and Fully Connected Layers

model_23.add(layers.Flatten())

model_23.add(layers.Dense(128, activation='relu'))    # First FC layer

model_23.add(layers.Dense(10, activation='softmax'))  #(output)

# 4. Compile model

model_23.compile(optimizer=SGD(momentum=0.9,learning_rate=0.01),
loss='sparse_categorical_crossentropy',metrics=['accuracy'])
```

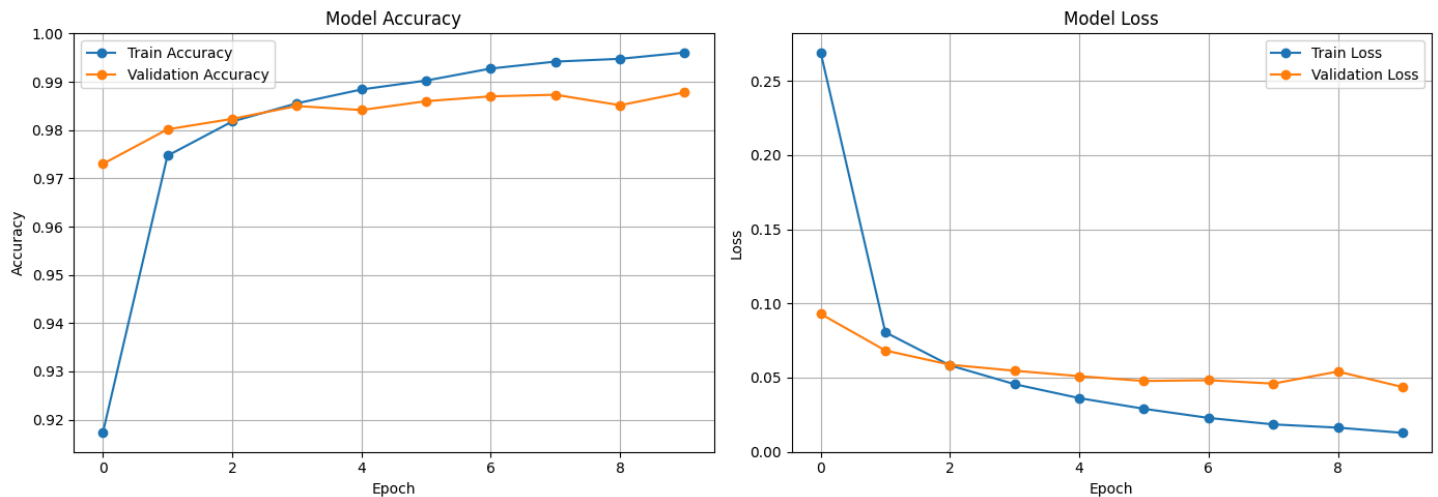
```
model_23.fit(X_train_conv, y_train, epochs=10, batch_size=32,  
validation_split=0.1, shuffle=True)
```

Accuracy in the first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	80.92%	957.30%
2	97.35%	98.02%
3	98.11%	98.23%
4	98.56%	98.50%
5	98.91%	98.42%

Final validation accuracy	98.78%
Test accuracy	98.84%
Number of parameters	225,034 parameters
Average train time	74.09 sec
Total test time	2.65 sec
Layers	1. Conv2D + ReLU 2. Pooling 3. Conv2D + ReLU 4. Pooling 5. Flatten 6. Dense (128) + ReLU 7. Output layer
Learning rate	0.01
Optimizer used	SGD

Observations



- The training and validation accuracies are closely aligned, which indicates good generalization with no signs of overfitting or underfitting. Both metrics show consistent improvement across epochs, further supporting the model's robustness.
- The loss is steadily decreasing for both training and validation sets, which is another excellent indicator of effective learning. This trend suggests that the model is optimizing well and learning meaningful patterns from the data.
- The total training time was approximately 740 seconds (~12 minutes), which is quite reasonable considering the architecture includes two convolutional layers, one fully connected layer, and was trained over 10 epochs. This setup strikes a good balance between computational efficiency and performance.

Conclusion

This journey of examining each parameter for its potential impact on model performance led us to a valuable insight: for a relatively simple dataset like MNIST, high accuracy (achieving 98.84%) can be attained with a minimal architecture. With just two convolutional layers and a single fully connected layer, the model achieves impressive results while maintaining low computational and memory demands. This demonstrates the power of convolutional layers and emphasizes that complexity is not always necessary to build an effective and resource-efficient model capable of generalizing well to unseen data.

Still, I believed there was room for improvement. To push the model's performance further, I experimented by changing the optimizer and introducing a dropout layer to reduce the risk of overfitting. As we will see in the final model, these modifications helped enhance the model's robustness and fine-tune its generalization ability—without compromising training time or computational efficiency.

Model(24)

```
model_24= models.Sequential()

# First Conv Layer + MaxPooling

model_24.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model_24.add(layers.MaxPooling2D((2,2)))

# Second Conv Layer + MaxPooling

model_24.add(layers.Conv2D(64, (3, 3), activation='relu'))

model_24.add(layers.MaxPooling2D((2,2)))

# Flatten and Fully Connected Layers

model_24.add(layers.Flatten())

model_24.add(Dropout(0.5))

model_24.add(layers.Dense(128, activation='relu'))    # First FC layer

model_24.add(layers.Dense(10, activation='softmax'))  #(output)

# 4. Compile model

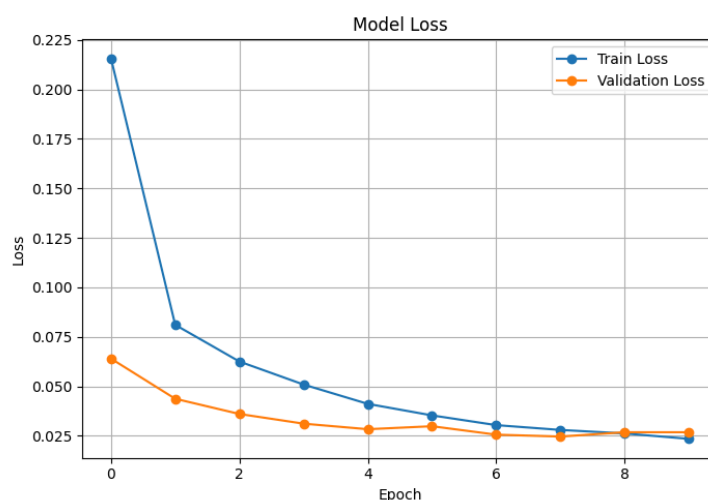
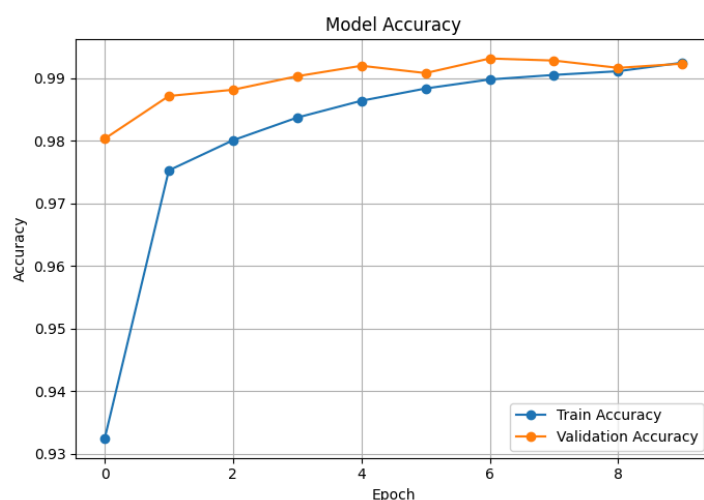
model_24.compile(optimizer=Adam(learning_rate=0.001),loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Accuracy in first 5 epochs

Epoch number	Training accuracy	Validation accuracy
1	85.81%	98.03%
2	97.48%	98.72%

3	97.91%	98.82%
4	98.33%	99.03%
5	98.72%	99.20%

Final validation accuracy	99.23%
Test accuracy	99.30%
Number of parameters	225,034 parameters
Average train time	73.48 sec
Total test time	5.29 sec
Layers	1. Conv2D + ReLU 2. Pooling 3. Conv2D + ReLU 4. Pooling 5. Flatten 6. Dropout (0.5) 7. Dense (128) + ReLU 8. Output layer
Learning rate	0.001 (It is the optimal learning rate for Adam)
Optimizer used	Adam (Adaptive Moment Estimation.)



Observations

- Validation accuracy showed early and steady improvement, reaching **99% by epoch 5**.
- Using **Adam with a learning rate of 0.01** initially decreased performance. Switching to **0.001** led to improvement, as the smaller rate allowed Adam to make more careful updates. Since Adam adjusts the learning rate individually for each parameter, using a high rate like 0.01 can cause the model to miss the optimal values.
- A **dropout layer (0.5)** was added before the fully connected layer to reduce a light overfitting observed in Model 23, where training accuracy slightly exceeded validation accuracy.
- **Training time remained ~735 seconds (~12.5 mins)**. Despite the additional dropout and optimizer switch, memory usage improved—dropout reduces the number of active neurons during training, lowering memory demand and computational overhead.

Final Thoughts

Starting with a simple CNN on the MNIST dataset, we were already hitting a strong 95% accuracy right out of the gate — a great start, no doubt. But good wasn't good enough. We wanted to push the boundaries and see just how far we could take it with minimal architecture and smart tuning.

Each tuning phase was like unlocking a new level of understanding. We didn't just tweak parameters blindly — every change led to a new insight about how CNNs behave and why certain choices matter more than others.

Eventually, we reached a **peak accuracy of 99.3%**. And sure, someone might say, "Eh, that's only a 4% gain from the start." But let's reframe that: if this model were diagnosing cancer, would you rather trust the one that leaves 5% uncertainty... or the one that says with 99.3% confidence, "You're good"? Yeah — I'm picking the 99.3% every time.

Through this journey, we discovered:

- Keeping a **reasonable number of epochs** and a **small batch size** gives the model time to actually *learn*, not just memorize.
- A **smart optimizer** like Adam with a **lower learning rate** makes your model smarter too — updating weights more precisely instead of chaotically jumping around.
- More layers? Not always better. In fact, **extra dense or convolutional layers** just ate up resources and often hurt performance. Bigger isn't always smarter.
- And **dropout**? Even when overfitting didn't scream for attention, introducing dropout gave us a noticeable edge — shaving off memory usage and squeezing out a few extra decimals of accuracy that made a real difference.

In the end, CNNs really are magic. They can take raw pixels and pull out meaningful insights, and when paired with thoughtful tuning, they deliver results that feel borderline *supernatural*. With just **two convolutional layers, one fully connected layer, and a pinch of patience**, we built a model that didn't just perform well — it performed smart.